

GlioTrace – Pseudocode Overview

INITIALIZATION

Take a list of microscopy stacks (.npz files) and a metadata table
Validate all inputs (file paths, channel assignments, sensitivity settings)
Match each stack file to its experiment metadata (time interval, patient, treatment group)
Mark all stacks as "not yet tracked"

IMAGE STABILIZATION (pre-processing, run once before analysis)

Read raw .tif microscopy frames folder by folder
For each experiment folder:
 Let user pick square ROIs on a preview image (or use full frame / coordinates from file)
 For each ROI:
 Stack all frames into a 3D array (height x width x time)
 For each consecutive pair of frames:
 Estimate the x/y drift using phase cross-correlation
 Shift all channels to correct for that drift
 Save stabilized stack as .npz

CELL DETECTION

For each frame in the green-channel stack:
 Subtract background (rolling-ball gaussian)
 Apply a Laplacian-of-Gaussian filter to find blob-like cells
 Find local maxima in the filtered image
 Threshold the image to find cell body regions
 Remove very large regions (not individual cells)
 For each remaining cell body region:
 Find the centroid → record as one cell position
 For each detected cell:
 Extract a small square image patch (61x61 pixels) from both green and red channels

CELL CLASSIFICATION (CNN)

For each detected cell patch:
 Feed the patch into a pre-trained CNN (MatlabNet, 4 conv layers + 3 FC layers)
 → Get probabilities for 6 morphology classes:
 Branching, Diffuse translocation, Crowded, Locomotion, Perivascular translocation, Round

 Feed the same patch into a second CNN (TME classifier)
 → Classify as: Microglia-colocalized / Vessel-associated / Non-

associated
(low-confidence predictions are reassigned to Non-associated)

CELL TRACKING (Kalman Filter)

Initialize one track per cell detected in the first frame
For each subsequent frame:
 Predict where each existing track should be (constant-velocity motion model)
 Measure actual detected cell positions
 Match predictions to measurements using the Hungarian algorithm (minimize distance)
 For matched cells: update track position using Kalman gain
 For unmatched detections: start a new track
 For unmatched tracks: keep for up to 2 frames using prediction only, then discard

After all frames:
 Stitch nearby track fragments together if their endpoints are within 15 pixels
 Assign a unique cell ID to each continuous track segment
 Output one row per (cell, timeframe) with position and classification labels

VASCULAR SEGMENTATION

For each frame in the red-channel (vasculature) stack:
 Apply a pre-trained U-Net segmentation network
 → Binary mask: vessel pixels vs background
 Clean the mask with morphological closing
 Skeletonize the vessel mask; remove tiny skeleton branches
 Record number of skeleton pixels as vessel length for this frame

For each tracked cell at each timepoint:
 Pool vessel masks over a ± 10 frame window (temporal smoothing)
 Find the nearest vessel pixel to each cell position (k-d tree nearest neighbour)
 Record that distance as "vascular_distance"

FEATURE CONSTRUCTION

For each tracked cell at each timepoint, compute:
 Speed = displacement between frames / time interval
 Direction = angle of movement
 Polarization = how aligned cells in the same ROI are moving (0=random, 1=parallel)
 Cosine similarity = how aligned this cell is with the ROI average direction
 TME label = simplified binary (microglia-colocalized yes/no)
 Vascular distance = log-transformed distance to nearest vessel

Sum green = total green signal in the frame (bulk tumor load)
adMAD = normalized frame-to-frame intensity change (activity measure)

For treatment experiments, also create interaction terms:
feature x is_treatment (e.g. speed_treat = speed for treated cells, 0 for controls)

Remove any cell tracks that are too short or have missing feature values

GLM-HMM FITTING (EM Algorithm)

Goal: fit a Hidden Markov Model where transition probabilities between states depend on cell features (via logistic regression = GLM)

Initialize:

6 hidden states (one per morphology class)

Uniform initial state distribution

Sticky transition matrix (high probability of staying in same state)

Repeat until convergence (or max iterations):

E-step (for each cell track):

Run forward-backward algorithm to compute:

$\gamma[t,k]$ = probability that cell is in state k at time t

$\xi[t,i,j]$ = probability of transitioning from state i to j at time t

M-step:

Update initial state distribution = average γ at $t=0$ across all tracks

Refit one logistic regression per state using ξ as soft training labels

→ "From state i , which features predict which next state?"

Output:

Transition matrix A (how often cells switch between states overall)

GLM models (how features modulate those transitions)

Gamma posteriors (per-cell per-timepoint state probabilities)

Viterbi path (most likely state sequence per track, via dynamic programming)

LEAVE-ONE-OUT (LOO) ANALYSIS

Goal: estimate uncertainty in the transition matrix using jackknife

resampling

1. Fit the full model on all matched treatment + control experiments
→ save
2. For each experimental set s :
 Refit the model leaving out set s → save
3. Compute jackknife statistics:
 global_a_mean = mean transition matrix across all L00 replicates
 global_a_var = jackknife variance of transition matrix
 For each feature:
 trans_diff = (transition at 90th percentile) - (transition at 10th percentile)
 trans_diff_mean = jackknife mean of that contrast
 trans_diff_var = jackknife variance of that contrast

VISUALIZATION

For each experiment/ROI:
 Load the raw fluorescence stack
 Generate an MP4 video:
 Each frame shows: [composite (green+red) | green only | morphology legend]
 Cell tracks are drawn as color-coded fading tails (color = morphology class)
 Dots mark current cell positions

 Optionally generate a comparison video:
 Left panel: cells colored by direct CNN classification
 Right panel: cells colored by Viterbi-decoded HMM state
 Side-by-side to compare CNN raw output vs HMM-smoothed output

SAVE / LOAD

Save a run to a folder:
 manifest.json – which pipeline stages have been completed
 config.json – all parameter settings
 subtable.pkl – stack file paths + metadata
 track_data.pkl – all tracked cell data
 data_feat.pkl – filtered feature table
 gammas.pkl – posterior state probabilities
 transition_matrix.pkl, pi.pkl – HMM parameters
 glm_models.joblib – fitted transition models

Load a run: read all the above files back and restore the GlioTrace object